

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

A robotics lab is testing **N mini robots over R rounds**. In each round, each robot sends a **two-digit signal code**. From each **valid code**, the **bigger digit** is the **power level**, and the **smaller digit** is the **stability level**. If the **difference between them is 5 or more**, the robot is **unstable**. Otherwise, it is **stable**. You are required to write a Java program that **takes R and N as input**, followed by the signal code of each robot in every round. For each code, **extract the two digits** and determine the **bigger and smaller digit**. If the **bigger digit is divisible by 3** and the **smaller digit is divisible by 7**, the code is **invalid**, and **input must be taken again** for that robot in that round. After each round, print the number of **stable and unstable robots**.

[Assume all inputs are valid two-digit integers from 10 to 99.]

| Sample Input                                                                                                                                                                                                | Sample Output                                                                                            | Explanation                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rounds (R) = 2<br>Robots (N) = 3<br>Round 1:<br>Robot 1 code = 42<br>Robot 2 code = 97<br>Robot 2 code = 81<br>Robot 3 code = 65<br>Round 2:<br>Robot 1 code = 54<br>Robot 2 code = 20<br>Robot 3 code = 31 | Round 1:<br>Stable robots: 2<br>Unstable robots: 1<br>Round 2:<br>Stable robots: 3<br>Unstable robots: 0 | Round 1:<br>42 → bigger = 4, smaller = 2 → difference = 2 → Stable<br>97 → bigger = 9, smaller = 7 → bigger divisible by 3 and smaller divisible by 7 → Invalid<br>81 → taken again for robot 2 → bigger = 8, smaller = 1 → difference = 7 → Unstable<br>65 → bigger = 6, smaller = 5 → difference = 1 → Stable<br>So, in Round 1:<br>Stable robots = 2<br>Unstable robots = 1 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

A game company is testing  $N$  mini game bots over  $R$  rounds. In each round, each bot sends a **two-digit performance code**. From each **valid code**, the **bigger digit** is the **attack level**, and the **smaller digit** is the **defense level**. If the **difference** between them is **less than 4**, the bot is **balanced**. Otherwise, it is **imbalanced**. You are required to write a Java program that takes  $R$  and  $N$  as **input**, followed by the **performance code** of each bot in every round. For each code, **extract the two digits** and determine the **bigger and smaller digit**. If the **bigger digit is divisible by 5** and the **smaller digit is divisible by 3**, the code is **invalid**, and **input must be taken again** for that bot in that round. After each round, print the number of **balanced and imbalanced bots**.

[Assume all inputs are valid two-digit integers from 10 to 99.]

| Sample Input                                                                                                                                                                              | Sample Output                                                                                            | Explanation                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rounds (R)= 2<br>Bots (N)= 3<br>Round 1:<br>Bot 1 code = 39<br>Bot 2 code = 11<br>Bot 3 Code = 81<br>Round 2:<br>Bot 1 code = 42<br>Bot 2 code = 53<br>Bot 2 code = 87<br>Bot 3 Code = 64 | Round 1:<br>Balanced bots: 1<br>Imbalanced bots: 2<br>Round 2:<br>Balanced bots: 3<br>Imbalanced bots: 0 | Round 2:<br>42 → bigger = 4, smaller = 2 → difference = 2 → Balanced<br>53 → bigger = 5, smaller = 3 → bigger is divisible by 5 and smaller is divisible by 3 → Invalid<br>87 → taken again for bot 2 → bigger = 8, smaller = 7 → difference = 1 → Balanced<br>64 → bigger = 6, smaller = 4 → difference = 2 → Balanced<br>So, in Round 2:<br>Balanced bots = 3<br>Imbalanced bots = 0 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

A cybersecurity team is checking encrypted message logs to identify hidden access codes. A valid code **starts with an uppercase letter (A-Z)** and **ends with an odd digit (0-9)**, including all characters in between. Once a valid code is found, **it is separated by #**, and the search for the next valid code must continue from the next character. Write a Java program that takes the encrypted message as input from the user, extracts all valid codes according to the rules, and displays them. If no valid code is found, "No valid code found." is displayed.

**You are only allowed to use .charAt() and .length() as built-in methods.**

| Sample Input    | Sample Output                                                                                                                                                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| abAxy3mnBz2pqC1 | Axy3#C1<br><br><b>Explanation</b><br>Axy3 → starts with A, ends with an odd digit 3 (valid)<br>Bz2 → starts with B, ends with an even digit 2 (invalid)<br>C1 → starts with C, ends with an odd digit 1 (valid)<br>Both valid codes are separated by #. |
| helloXabcYz     | No valid code found.                                                                                                                                                                                                                                    |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

A research team is scanning DNA samples to identify hidden sample labels. A valid label **starts with an even digit (0-9)** and **ends with a lowercase letter (a-z)**, including all characters in between. Once a valid label is found, **it is separated by @**, and the search for the next valid code must continue from the next character. Write a Java program that takes a DNA sequence as input from the user, extracts all valid labels according to the rules, and displays them. If no valid label is found, "No valid label found." is displayed.

**You are only allowed to use .charAt() and .length() as built-in methods.**

| Sample Input    | Sample Output                                                                                                                                                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2ADxBc5mnDe6qfs | 2ADx@6q<br><br><b>Explanation</b><br>2ADx → starts with an even digit 2, ends with x (valid)<br>5m → starts with an odd digit 5, ends with m (invalid)<br>6q → starts with an even digit 6, ends with q (valid)<br>Both valid codes are separated by @. |
| 135BCD79EFG     | No valid label found.                                                                                                                                                                                                                                   |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

Write a Java program that counts the unique (distinct) elements in a given array and then creates a new array containing only those unique elements. Next, your program sorts the new array **in place** so that elements at **even indices** are in **ascending order** and elements at **odd indices** are in **descending order**. Finally, it displays the count of unique elements and the sorted array.

| Given                                                                                                                                                                                              | Expected Output                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>int [] marks = {9, -5, 7, 9, -5, 5};</code>                                                                                                                                                  | Unique count: 4<br>Sorted array: 7 5 9 -5 |
| <b>Explanation:</b><br>The unique elements are 9, -5, 7, and 5, so the unique count is 4.<br>Even indices are sorted in ascending order: 7, 9<br>Odd indices are sorted in descending order: 5, -5 |                                           |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

Write a Java program that counts the unique (distinct) elements in a given array and then creates a new array containing only those unique elements. Next, your program sorts the new array **in place** so that elements at **even indices** are in **descending order** and elements at **odd indices** are in **ascending order**. Finally, it displays the count of unique elements and the sorted array.

| Given                                                                                                                                                                                            | Expected Output                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <code>int [] scores= {4, 2, 6, 1, 4, 1};</code>                                                                                                                                                  | Unique count: 4<br>Sorted array: 6 1 4 2 |
| <b>Explanation:</b><br>The unique elements are 4, 2, 6, and 1, so the unique count is 4.<br>Even indices are sorted in descending order: 6, 4<br>Odd indices are sorted in ascending order: 1, 2 |                                          |